One Identity Manager 9.3

HTML5 Development Guide

Copyright 2024 One Identity LLC.

ALL RIGHTS RESERVED.

This guide contains proprietary information protected by copyright. The software described in this guide is furnished under a software license or nondisclosure agreement. This software may be used or copied only in accordance with the terms of the applicable agreement. No part of this guide may be reproduced or transmitted in any form or by any means, electronic or mechanical, including photocopying and recording for any purpose other than the purchaser's personal use without the written permission of One Identity LLC .

The information in this document is provided in connection with One Identity products. No license, express or implied, by estoppel or otherwise, to any intellectual property right is granted by this document or in connection with the sale of One Identity LLC products. EXCEPT AS SET FORTH IN THE TERMS AND CONDITIONS AS SPECIFIED IN THE LICENSE AGREEMENT FOR THIS PRODUCT, ONE IDENTITY ASSUMES NO LIABILITY WHATSOEVER AND DISCLAIMS ANY EXPRESS, IMPLIED OR STATUTORY WARRANTY RELATING TO ITS PRODUCTS INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTY OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, OR NON-INFRINGEMENT. IN NO EVENT SHALL ONE IDENTITY BE LIABLE FOR ANY DIRECT, INDIRECT, CONSEQUENTIAL, PUNITIVE, SPECIAL OR INCIDENTAL DAMAGES (INCLUDING, WITHOUT LIMITATION, DAMAGES FOR LOSS OF PROFITS, BUSINESS INTERRUPTION OR LOSS OF INFORMATION) ARISING OUT OF THE USE OR INABILITY TO USE THIS DOCUMENT, EVEN IF ONE IDENTITY HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES. One Identity makes no representations or warranties with respect to the accuracy or completeness of the contents of this document and reserves the right to make changes to specifications and product descriptions at any time without notice. One Identity does not make any commitment to update the information contained in this document.

If you have any questions regarding your potential use of this material, contact:

One Identity LLC.
Attn: LEGAL Dept
4 Polaris Way
Aliso Viejo, CA 92656

Refer to our website (http://www.OneIdentity.com) for regional and international office information.

Patents

One Identity is proud of our advanced technology. Patents and pending patents may apply to this product. For the most current information about applicable patents for this product, please visit our website at http://www.OneIdentity.com/legal/patents.aspx.

Trademarks

One Identity and the One Identity logo are trademarks and registered trademarks of One Identity LLC. in the U.S.A. and other countries. For a complete list of One Identity trademarks, please visit our website at www.OneIdentity.com/legal/trademark-information.aspx. All other trademarks are the property of their respective owners.

Legend

WARNING: A WARNING icon highlights a potential risk of bodily injury or property damage, for which industry-standard safety precautions are advised. This icon is often associated with electrical hazards related to hardware.

CAUTION: A CAUTION icon indicates potential damage to hardware or loss of data if instructions are not followed.

One Identity Manager HTML5 Development Guide
Updated - 13 December 2024, 21:32

For the most recent documents and product information, see Online product documentation.

Contents

About this guide **4**

Architecture of One Identity Manager HTML applications **5**

Developing HTML applications with the GitHub repository **6**

Angular workspace architecture 6

Setting up a working environment for the GitHub repository 8

Accessing the API 9

Creating and editing your own HTML applications 10

- Merging One Identity Manager updates 10
- Customizing libraries 11
- Adding plugins 11
- Checking for missing translations 13
- Registering HTML applications 13
- Compiling and deploying HTML applications 14

Debugging 16

- Hosting API Servers locally 16
- Debugging with plugins 17

About us **19**

Contacting us 19

Technical support resources 19

1

About this guide

This guide shows you to view One Identity Manager HTML applications as code and how to understand their internal functionality. You will discover how to create your own HTML applications and implement them.

You can use existing HTML applications from a GitHub repository as templates for this (see Developing HTML applications with the GitHub repository on page 6).

Available documentation

The online version of One Identity Manager documentation is available in the Support portal under Technical Documentation. You will find videos with additional information at www.YouTube.com/OneIdentity.

2

Architecture of One Identity Manager HTML applications

The HTML applications are structured as nodeJS applications that use the **Angular** framework. Generally, any HTML applications that can be compiled as nodeJS applications are supported.

HTML applications use the API Client to communicate with the One Identity Manager API. The API Client is an npm library that is automatically generated and stored to the database during API compilation. The API Client controls all network access on the API Server.

For more information about API development, see the *One Identity Manager API Development Guide*.

3

Developing HTML applications with the GitHub repository

You can develop your own HTML applications using the source code of a default HTML applications as a template.

The source code of the standard HTML applications is available in a GitHub repository.

Angular workspace architecture

The GitHub repository contains the source code for the HTML applications in One Identity Manager. It is a monorepo that contains the Angular workspace, which consists of applications and libraries.

The Angular workspace integrates the Nx tool. This tool makes it easier to work with monorepo environments. Nx is installed automatically. One Identity recommends you use Nx to achieve better build speed and simplify the management of project dependencies. For more information, see here.

Note the following when using Nx:

- The Nx workspace is defined in the `nx.json` file.
- Each project has its own `project.json` declaration file. Nx uses these files to manage dependencies between projects and to determine the compilation order.

> TIP: To visualize the dependencies between projects, run the `nx graph` command from the command line.

Each Angular library and application belongs to a folder in the `projects` directory. The Angular workspace is defined in the `angular.json` file.

Table 1: Angular libraries

Name	Type	Dependencies within the workspace
qbm	Angular library	none
qer	Angular library	qbm
dpr	Angular library	qbm
apc	Angular plugin library	qbm, qer
hds	Angular plugin library	qbm, qer
olg	Angular plugin library	qbm, qer
rmb	Angular plugin library	qbm, qer
rps	Angular plugin library	qbm, qer
sac	Angular plugin library	qbm, qer
qam	Angular plugin library	qbm, qer
tsb	Angular plugin library	qbm, qer
att	Angular plugin library	qbm, qer
rms	Angular plugin library	qbm, qer
aad	Angular plugin library	qbm, qer, tsb
aob	Angular plugin library	qbm, qer
uci	Angular plugin library	qbm, qer
cpl	Angular plugin library	qbm, qer
o3t	Angular plugin library	qbm, qer, tsb

Name	Type	Dependencies within the workspace
pol	Angular plugin library	qbm, qer

Each Angular library belongs to the One Identity Manager module of the same name.

An Angular library behaves like a regular compile-time dependency.
A plugin library is loaded dynamically at runtime. This is specified in the plugin's `imx-plugin-config.json` files.

Table 2: Angular applications

Name	Description	Project type	Static dependencies
qbm-app-landingpage	API Server landing page and server management	Angular application	qbm
qer-app-portal	Web Portal	Angular application	qbm, qer
qer-app-operationssupport	Operations Support Web Portal	Angular application	qbm, qer
qer-app-pwdportal	Password Reset Portal	Angular application	qbm, qer

Setting up a working environment for the GitHub repository

In this section, you will discover how to set up your working environment for using the GitHub repository of default HTML applications. This will allow you to develop your own web applications.

Prerequisites:

- You have a valid GitHub account (see https://github.com/).

To set up your working environment

1. Request access to the GitHub repository.
2. Create a branch of the corresponding branch of the GitHub repository (see https://docs.github.com/en/get-started/quickstart/fork-a-repo).
3. Install npm (see https://docs.npmjs.com/downloading-and-installing-node-js-and-npm).

4. In your local repository, change to the `imxweb` directory and run the following command in from the command line:

```
npm install
```

Accessing the API

The Typescript API client is used to access the API.

For each One Identity Manager module that provides API services, an `imx-api-<module name>.tgz` Angular library exists as an NPM package in the `imx-modules` folder of the repository.

> NOTE: The API client libraries are not dependent on Angular and thus can be used from any JavaScript program.

The TypeScript API clients consist of several parts:

- **Endpoint-based methods**:

  The **V2Client** class of the API client contains one method for each API endpoint. The method name is generated with the following format:

  ```
  <URL path of the API endpoint>_<HTTP method>
  ```

  Example

  The **GET portal/serviceitems** method becomes the **portal_serviceitems_get** typescript method.

- **Entity-based methods**:

  The **TypedClient** class of the API client contains a wrapper class for each entity-based API method that allows entities to be loaded and stored.

  Example of methods for interactive entities

  For the **portal/serviceitems/interactive** method there is the **TypedClient.PortalServiceItemsInteractive** property of the **PortalServiceItemsInteractive** wrapper class type.

  Depending on the scope of operations supported by an API method, the following methods are available for the wrapper class:

- The **createEntity** method is used to create a new entity.
- The **Get_byid** method is used to load an interactive entity from the API Server. The API Server only supports loading a single object per query from the database as an interactive entity. The primary key values of the object must be specified as method parameters.
- The **Put** and **Post** methods are used to store entities with the PUT operation. These methods must not be called directly, but are controlled by the **commit()** method of the IEntity interface.

Encapsulation as a service

Access to the API is encapsulated in a separate Angular service for each Angular library. This Angular service can be imported into custom classes:

- In the **qbm** Angular library, the service is called **imx_SessionService**.
- In the **qer** Angular library, the service is called **QerApiService**.
- In all other Angular libraries, the service is called **ApiService**.

Creating and editing your own HTML applications

To create and edit your own HTML applications, you can modify Angular libraries and add plugins to the API Server.

TIP: To illustrate this, One Identity provides the **custom-app** HTML5 application as an example in the Angular workspace.

Merging One Identity Manager updates

As soon as One Identity releases product updates for One Identity Manager, you can apply relevant changes to your customized HTML applications.

NOTE: You customized HTML applications are not automatically updated. You are responsible for compiling and deploying up-to-date versions of an HTML application.

To apply changes from a One Identity Manager update to your HTML applications

1. Take the code changes from the GitHub repository of default HTML applications and apply them to your HTML application (see https://docs.github.com/pull-

requests/collaborating-with-pull-requests/working-with-forks/merging-an-upstream-repository-into-your-fork).

2. Compile your HTML application (see Compiling and deploying HTML applications on page 14) and fix any compilation errors that may occur.
3. Check the One Identity Manager release notes for changes to the default API or data formats, and verify that your HTML applications still work properly.
4. Deploy the new version of your HTML application (see Compiling and deploying HTML applications on page 14).

Customizing libraries

If you modify Angular library code, you must compile all the Angular libraries and create and deploy custom versions of all Angular applications that will use the modified Angular library. This applies regardless of which Angular libraries contain changed code.

For example, if you modify the **qer** Angular library, you must also compile all the Angular libraries and the **qer-app-portal**, **qer-app-operationsupport**, and **qer-app-pwdportal** Angular applications because all of these applications contain the **qer** Angular library.

If you are using Nx, Nx takes over the management of dependencies between libraries. Use the `npm run nx:build-all` command to compile all projects in the specified sequence.

Adding plugins

Plugins are Angular libraries that are dynamically loaded at runtime. The plugins are managed by the API Server. Plugins are automatically detected by the API Server by searching the program directory for files named `imx-plugin-config.json`.

The following sample file specifies that the Angular plugin library `ccc` should be loaded into the **qer-app-portal application**. The name of the Angular module to be instantiated is **CustomConfigModule**.

```
{
   "qer-app-portal":[
      {
         "Container":"ccc",
         "Name":"CustomConfigModule"
```

```
        }
    ]
}
```

To add a plugin

1. On the API Server, create the file `imxweb\<Angular plugin library name>\imx-plugin-config.json` with the following content:

```
{
   "<Name of the HTML application>":[
      {
         "Container":"<Name of the Angular plugin library>",
         "Name":"<Name of the Angular module>"
      }
   ]
}
```

2. Import the file into your One Identity Manager database using the Software Loader and assign it to the **API Server** machine role. For more information on importing files using the Software Loader, see the *One Identity Manager Operational Guide*.
3. (Optional) To check if the HTML application loads the plugin correctly, open the **`<API Server URL>/imx/applications`** URL and check that the corresponding plugin appears at the HTML application position in the list.

To add a plugin using Nx

1. Create a new project using the Nx command line program (`nx generate lib <project name>`).
2. In the newly created project, open the `projects/<project name>/project.json` file and add all dependencies.
3. Perform the following actions for each HTML application that is to use this plugin:
   a. Open the application's `project.json` file (such as `projects/qer-app-portal/project.json`) with a text editor.
   b. Add a library with the name of your new project under `prebuild > dependsOn > projects`.

      For example, if your project is called **CCC**, you must change the line as follows:

```
"projects": ["qer", "aad", "aob", "apc", "att", "CCC"]
```

c. Save the file.

Checking for missing translations

You can check HTML applications for missing user interface translations using the ImxClient command line program. For more information about the ImxClient command line program, see the *One Identity Manager API Development Guide*.

TIP: Translatable text is marked in the source code with the `#LDS#` prefix.

To check an HTML application for missing translations

1. Start the ImxClient command line program.
2. In the folder you want to check for missing translations, run the **check-translations** command.

   This creates a report. The report shows you any files containing texts that have not yet been translated or have not been translated in full.
3. (Optional) To add translation keys and translations, use the Designer program. For more information about translations, see the *One Identity Manager Configuration Guide*.

Registering HTML applications

To deploy new HTML applications for use and display them on the API Server's home page, you must add the HTML applications to the database.

To add an HTML application to the database

1. Start the Designer program.
2. Connect to the relevant database.
3. In the navigation, click the **Base data** > **Security settings** > **HTML applications** category.
4. On the menu bar, click **(Create a new object**).
5. Click the new entry in the list.
6. In the **Properties** view, enter the HTML application data in the respective fields. Enter at least the following information:

- **Display name**: Enter a name for the HTML application.
- **HTML application**: Enter the path `CCC/<name of your HTML application>`.
- **Precompiled**: Set the value to `True`.

Compiling and deploying HTML applications

To deploy an HTML application, compile the HTML application and provide the package as a ZIP file.

To compile and deploy an HTML application via the API Server

1. Start a command line program.
2. Change to the Angular workspace directory.
3. Compile all the libraries that are loaded by the HTML application.

   If you are using Nx, run the following command:

   ```
   npx build <application name>
   ```

   NOTE: Angular libraries do not have to be deployed separately. HTML applications contain all the necessary libraries.
4. Run the following command:

   ```
   ng build <application name>
   ```

5. Change to the directory with the compilation (normally `dist/<application name>`) and add all the files and subfolders to a new ZIP file with the name `Html_<application name>.zip`.
6. Copy the ZIP file to the `imxweb\custom` subfolder of your workspace.

   TIP: If the folder does not exist, create it.
7. Import the ZIP file into your One Identity Manager database using the Software Loader and assign it to the **Development and Testing** machine role. For more information on importing files using the Software Loader, see the *One Identity Manager Operational Guide*.
8. Copy the ZIP file to the `bin\imxweb\custom` subfolder of your IIS installation.

   TIP: If the folder does not exist, create it.
9. Import the ZIP file into your One Identity Manager database using the Software Loader and assign it to the **Business API Server** machine role.
10. (Optional) To check whether your Angular package (and therefore your HTML application) is loaded correctly, check that the corresponding packages are labeled as **Customized package** in the Administration Portal. For more information about displaying packages, see the *One Identity Manager Web Application Configuration Guide*.

To compile and deploy an HTML application without the API Server

1. Use the Software Loader program to export the API Server file structure from your One Identity Manager database to a local folder. Select the **Server**, **Web**, **Business API Server**, and **SCIM Provider** machine roles. For more information on exporting files with the Software Loader, see the *One Identity Manager Operational Guide*.
2. Start a command line program.
3. Change to the Angular workspace directory.
4. Compile all the libraries that are loaded by the HTML application.

   If you are using Nx, run the following command:

   ```
   npx build <application name>
   ```

   NOTE: Angular libraries do not have to be deployed separately. HTML applications contain all the necessary libraries.
5. Run the following command:

   ```
   ng build <application name>
   ```
6. Change to the directory with the compilation (normally `dist/<application name>`) and add all the files and subfolders to a new ZIP file with the name `Html_<application name>.zip`.
7. Copy the ZIP file to the `imxweb\custom` subfolder of your workspace.

   TIP: If the folder does not exist, create it.
8. Import the ZIP file into your One Identity Manager database using the Software Loader and assign it to the **Development and Testing** machine role. For more information on importing files using the Software Loader, see the *One Identity Manager Operational Guide*.
9. Copy the ZIP file to the `bin\imxweb\custom` subfolder of the folder created in the first step.

   TIP: If the folder does not exist, create it.
10. Import the ZIP file into your One Identity Manager database using the Software Loader and assign it to the **Business API Server** machine role.
11. (Optional) To check whether your Angular package (and therefore your HTML application) is loaded correctly, check that the corresponding packages are labeled as **Customized package** in the Administration Portal. For more information about displaying packages, see the *One Identity Manager Web Application Configuration Guide*.

Debugging

Running and debugging HTML applications is possible with the Angular CLI toolchain's standard tools.

For example, you can use the `ng serve qer-app-portal` command to debug the Web Portal HTML application.

To debug an HTML application

1. Host the API Server locally (see Hosting API Servers locally on page 16).
2. Start a command line program.
3. Change to the Angular workspace directory.
4. Run the following command:

   ```
   npm run start <HTML application>
   ```

   This starts a web server that is accessible by default under `http://localhost:4200` and hosts the HTML application.
5. Start debugging in an appropriate development environment (for example, Visual Studio Code).

Hosting API Servers locally

To debug and develop an HTML application you need an instance of an API Server for connecting HTML applications. To do this, you can host an API Server locally.

NOTE: HTML applications connect with the API Server through the URL defined in the HTML application's `environment.ts` file. The default URL that runs under a locally hosted API Server, is `http://localhost:8182`.

To host an API Server locally

1. Start a command line program.
2. Switch to the One Identity Manager installation directory.
3. Run the following command:

   ```
   imxclient.exe run-apiserver -B
   ```

   NOTE: This command does not use the results of a local compilation from your development environment but the compiled state of the HTML application from the

delivery version. For more information about how you call an HTML application from your local development environment, see Debugging with plugins on page 17.

Debugging with plugins

You can also debug with plug-ins. Debugging with plug-ins only works if the local API Server can find the plug-in.

To debug a static Angular library

1. Host the API Server locally (see Hosting API Servers locally on page 16).
2. Start a command line program.
3. Change to the Angular workspace directory.
4. Run the following command:

```
npm run build:watch <Angular library>
```

5. Start another command line program.
6. Change to the Angular workspace directory.
7. Run the following command:

```
npm run start <HTML application>
```

   This starts a web server that is accessible by default under `http://localhost:4200` and hosts the HTML application.

8. Start debugging in an appropriate development environment (for example, Visual Studio Code).

To debug an Angular plug-in library

1. Host the API Server locally (see Hosting API Servers locally on page 16).
2. Start a command line program.
3. Change to the Angular workspace directory.
4. Run the following command:

```
npm run build:watch:dynamic <Angular plug-in library>
```

5. Start another command line program.
6. Change to the Angular workspace directory.
7. Run the following command:

```
npm run start <HTML application>
```

This starts a web server that is accessible by default under `http://localhost:4200` and hosts the HTML application.

8. Start debugging in an appropriate development environment (for example, Visual Studio Code).

About us

One Identity solutions eliminate the complexities and time-consuming processes often required to govern identities, manage privileged accounts and control access. Our solutions enhance business agility while addressing your IAM challenges with on-premises, cloud and hybrid environments.

Contacting us

For sales and other inquiries, such as licensing, support, and renewals, visit https://www.oneidentity.com/company/contact-us.aspx.

Technical support resources

Technical support is available to One Identity customers with a valid maintenance contract and customers who have trial versions. You can access the Support Portal at https://support.oneidentity.com/.

The Support Portal provides self-help tools you can use to solve problems quickly and independently, 24 hours a day, 365 days a year. The Support Portal enables you to:

- Submit and manage a Service Request
- View Knowledge Base articles
- Sign up for product notifications
- Download software and technical documentation
- View how-to videos at www.YouTube.com/OneIdentity
- Engage in community discussions
- Chat with support engineers online
- View services to assist you with your product